TUGAS MATA KULIAH STRATEGI OPTIMASI KOMPUTASI AWAN

TASK 6



Oleh:

5027241050 Daniswara Fausta Novanto

5027241103 Ni`mah Fauziyyah Atok

5027241042 Abiyyu Raihan Putra Wikanto

5027241082 Muhammad Rafi` Adly

5027231043 Adi Satria Pangestu

DEPARTEMEN TEKNOLOGI INFORMASI INSTITUT TEKNOLOGI SEPULUH NOPEMBER SURABAYA 2026

Implementasi Algoritma Heuristik Minimum Execution Time (MET) pada Cloud Nyata Microsoft Azure

Ringkasan Eksekutif

Proyek ini menerapkan Minimum Execution Time (MET) pada cloud nyata Microsoft Azure untuk menjadwalkan pemrosesan 100 gambar JPEG ke empat VM worker. Round Robin (RR) digunakan sebagai pembanding. Pada run baseline 4 worker, MET menyelesaikan pekerjaan dalam 11,4768 detik, sedangkan RR membutuhkan 5,4666 detik. Throughput RR mencapai 18,2928 task/detik dibandingkan 8,7132 task/detik pada MET. MET menempatkan seluruh 100 task pada vm-worker-1; RR membagi 25 task ke setiap worker. Hasil ini menunjukkan bahwa pada implementasi dan workload yang diuji, MET mengalami penumpukan task karena tidak memperhitungkan antrean atau beban VM.

Hasil masih memiliki keterbatasan penting. Label load_300 ternyata tetap memproses 100 task karena manifest hanya berisi 100 gambar. Uji resize tidak memiliki dokumentasi ukuran VM sebelum/sesudah. Uji failover Azure menghentikan vm-worker-4, bukan VM yang menjadi target MET. Run baseline yang dipakai hanya satu run per algoritma; data berlabel load_300 bukan pengujian beban independen. Oleh sebab itu, kesimpulan tentang skalabilitas, dampak resize, dan failover target harus dibatasi.

Bab 1. Pendahuluan dan Arsitektur Azure

1.1 Latar Belakang dan Tujuan

Pada tugas sebelumnya, MET dievaluasi melalui simulasi CloudSim dengan 20 VM heterogen dan workload GoCJ. Pada proyek ini, MET dijalankan pada cloud nyata dengan workload pemrosesan gambar. Tujuannya adalah menguji perilaku algoritma dalam sistem terdistribusi yang benar-benar melakukan pemrosesan, serta membandingkannya dengan Round Robin menggunakan makespan, throughput, waktu tunggu, Degree of Imbalance (DI), dan utilisasi berbasis waktu sibuk.

1.2 Arsitektur

Arsitektur menggunakan satu VM master untuk menjalankan master.py dan scheduler.py, serta empat VM worker untuk memproses gambar. Repo mengelompokkan vm-worker-1 dan vm-worker-2 sebagai Host A, serta vm-worker-3 dan vm-worker-4 sebagai Host B. Pengelompokan ini adalah label arsitektur proyek; keberadaan dua host fisik Azure perlu dibuktikan melalui konfigurasi Azure yang sebenarnya, karena Azure umumnya tidak menampilkan pemetaan server fisik pelanggan.

VM	Peran	vCPU yang dilaporkan worker	Ukuran SKU Azure
vm-master	Master / scheduler	1	2
vm-worker-1	Worker	2	1 of 2
vm-worker-2	Worker	2	2
vm-worker-3	Worker	2	2
vm-worker-4	Worker	2	2

Bab 2. Dataset dan Worker

2.1 Dataset

Dataset dibuat secara sintetis oleh dataset_generator.py dengan seed 42. Manifest berisi 100 gambar JPEG: 30 gambar berukuran 640×480, 30 berukuran 1280×720, 30 berukuran 1920×1080, dan 10 berukuran 2560×1440. Panjang task abstrak dihitung sebagai length = lebar × tinggi / 10000. Total length dataset adalah 13.593,60. Label load_300 pada hasil tidak menunjukkan 300 gambar: manifest yang tersedia tetap 100 baris, sehingga --limit 300 hanya memuat 100 task.

2.2 Worker

worker.py menyediakan layanan FastAPI pada port 8000. Endpoint GET /health mengembalikan status, identitas VM, dan jumlah core. Endpoint POST /process menerima gambar dan menjalankan pipeline Pillow berupa konversi RGB, Gaussian blur, resize LANCZOS pada skala 0,5, unsharp mask, dan penyimpanan JPEG kualitas 85. Pipeline dijalankan dua kali; pass kedua bekerja pada gambar yang telah diperkecil. Worker memakai ProcessPoolExecutor berdasarkan jumlah vCPU yang dilaporkan.

Bab 3. Algoritma dan Metrik

3.1 Minimum Execution Time (MET)

MET menghitung estimasi waktu ET(i,j) = length_i / capacity_j untuk setiap task i dan VM j, lalu memilih VM dengan estimasi waktu terkecil. Implementasi memakai toleransi seri 5%; jika kapasitas berada dalam toleransi tersebut, VM yang muncul lebih dahulu dalam daftar dipilih. Karena MET tidak memperhitungkan antrean atau beban yang sudah diberikan, task dapat terus menumpuk pada satu VM.

3.2 Round Robin

Round Robin membagikan task secara bergiliran ke worker yang tersedia. Pada baseline 100 task dan empat worker, RR membagikan 25 task ke setiap worker. Pembagian jumlah task sama tidak berarti beban total sama karena gambar memiliki ukuran dan length berbeda.

3.3 Definisi Metrik

Metrik	Definisi yang dipakai
Makespan	Waktu sampai task terakhir selesai.
Throughput	Jumlah task / makespan, satuan task/detik.
avg_wait_s	Rata-rata send_s, yaitu waktu relatif ketika task mulai dikirim dari master; bukan waktu antre murni yang diukur terpisah.
DI	(Tmax − Tmin) / Tavg, dengan Tj berupa finish_s terbesar pada VM; VM tanpa task diberi nilai 0.
Utilisasi rata-rata	Rata-rata util_per_vm; util_per_vm = jumlah exec_s / (cores × makespan) × 100%. Ini bukan CPU% dari sistem operasi.
Waktu respons	finish_s karena semua task dianggap tersedia pada t0.
Waktu layanan	finish_s − send_s.

Self-test scheduler.py dilaporkan lulus dan menghasilkan DI = 20,0 untuk model 20 VM, konsisten dengan hasil simulasi terdahulu. Ini adalah uji logika scheduler, bukan pengukuran performa Azure.

Bab 4. Master dan Alur Eksekusi

master.py membaca manifest dan konfigurasi worker, memeriksa kesehatan worker melalui /health, mengkalibrasi kapasitas, memanggil scheduler untuk menentukan tujuan task, mengirim gambar ke worker, menangani worker yang tidak sehat, menghitung metrik, lalu menulis summary.csv dan tasks_<ALGO>_<timestamp>.csv. Kalibrasi menggunakan gambar 1920×1080 dengan length 207,36. Kapasitas dihitung dari cores × 207,36 dibagi waktu dinding yang diamati master. Karena waktu dinding dapat mencakup transfer dan latensi jaringan, kapasitas ini tidak selalu sama dengan kecepatan pemrosesan CPU murni.

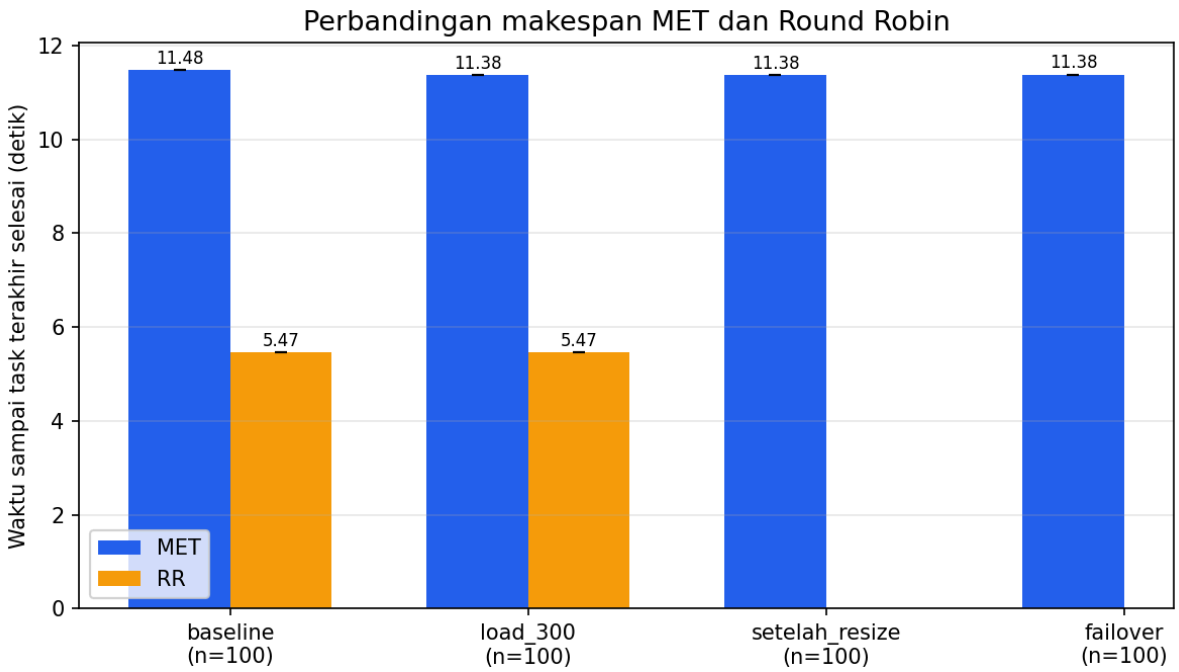
summary.csv menyimpan satu baris per run; tasks_*.csv menyimpan data per task. File tasks dipasangkan ke summary melalui timestamp. Empat run awal menggunakan tiga worker dan tidak dicampur ke perbandingan baseline empat worker.

Bab 5. Pengujian, Hasil, Analisis, dan Kesimpulan

5.1 Hasil Baseline Azure

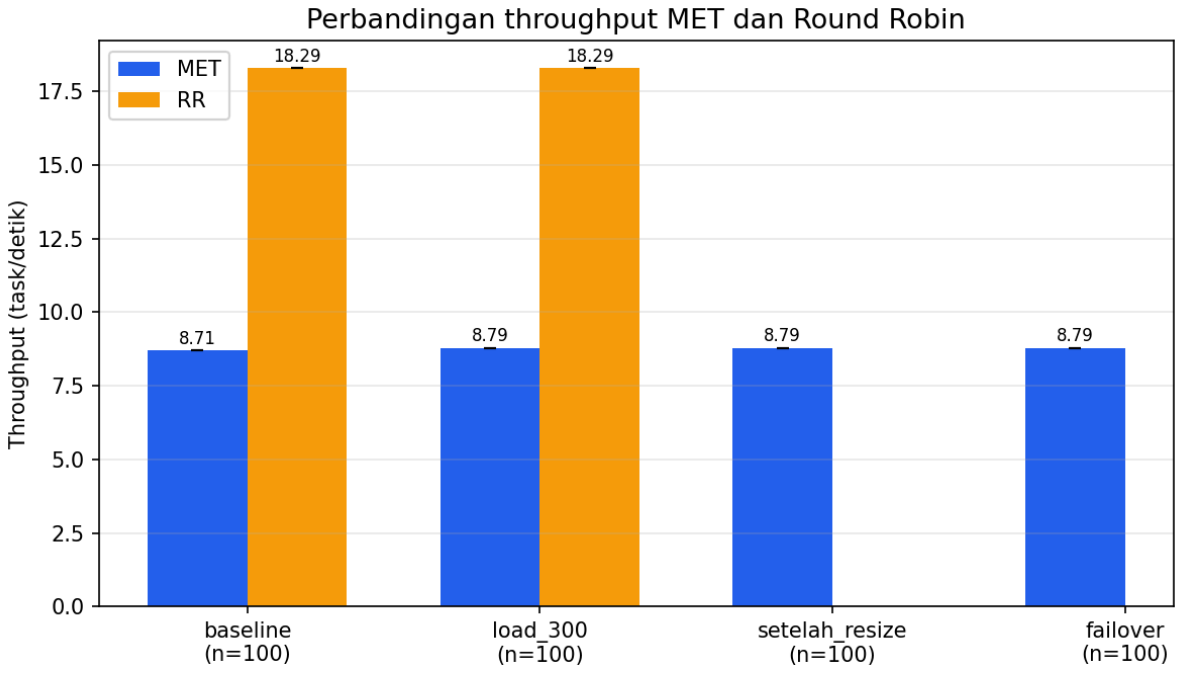
Metrik	MET	Round Robin	Interpretasi ringkas
Jumlah task	100	100	Beban sama.
Makespan (s)	11,4768	5,4666	RR sekitar 2,10 kali lebih cepat.
Waktu tunggu rata-rata (s)	5,5845	1,9266	Nilai avg_wait_s MET sekitar 2,90 kali RR.
Throughput (task/s)	8,7132	18,2928	RR sekitar 2,10 kali throughput MET.
DI	4,0000	0,6042	MET mencapai kondisi timpang ekstrem untuk empat VM.
Utilisasi rata-rata (%)	23,78	42,51	MET hanya memakai satu dari empat worker.
Respons rata-rata (s)	5,8124	2,2599	Dihitung dari finish_s task CSV.
Respons median (s)	5,8390	2,0868	Dihitung dari finish_s task CSV.
Waktu layanan rata-rata (s)	0,2278	0,3334	finish_s − send_s; definisi operasional sesuai handoff.

Sumber: run baseline empat worker dengan timestamp 2026-10-29 02:03:04 untuk MET dan 2026-10-29 02:03:11 untuk RR. Tiap algoritma hanya memiliki satu run baseline yang dipakai di tabel; tidak ada dasar untuk uji signifikansi statistik.



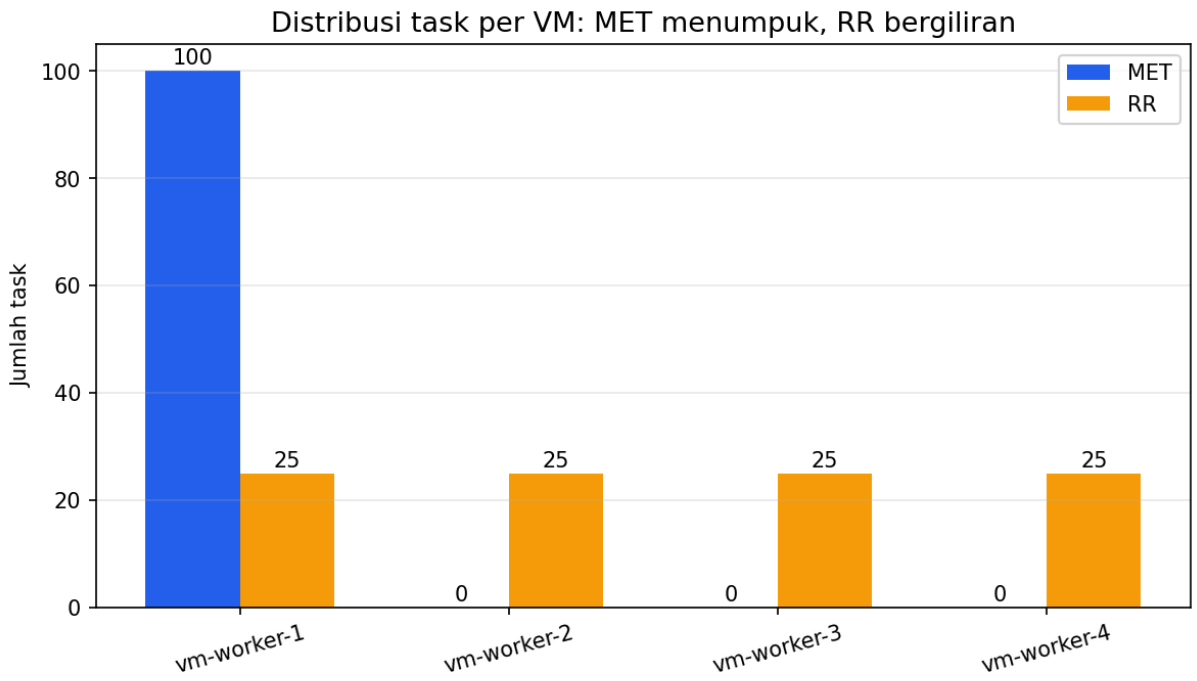
Data Azure/lokal sesuai run terpilih; batang = rata-rata run per label dan algoritma. Error bar = SD bila n > 1. Worker dihitung dari data tiap run.

Gambar 5.1. Perbandingan makespan MET dan Round Robin.

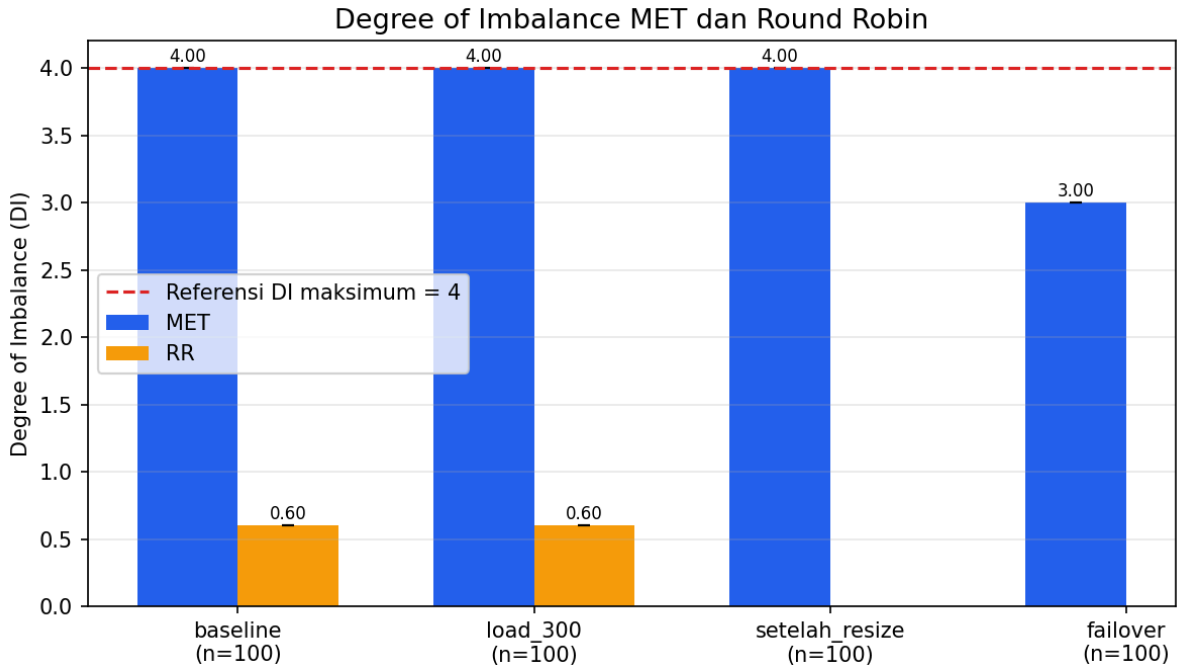


Data Azure/lokal sesuai run terpilih; batang = rata-rata run per label dan algoritma. Error bar = SD bila n > 1. Worker dihitung dari data tiap run.

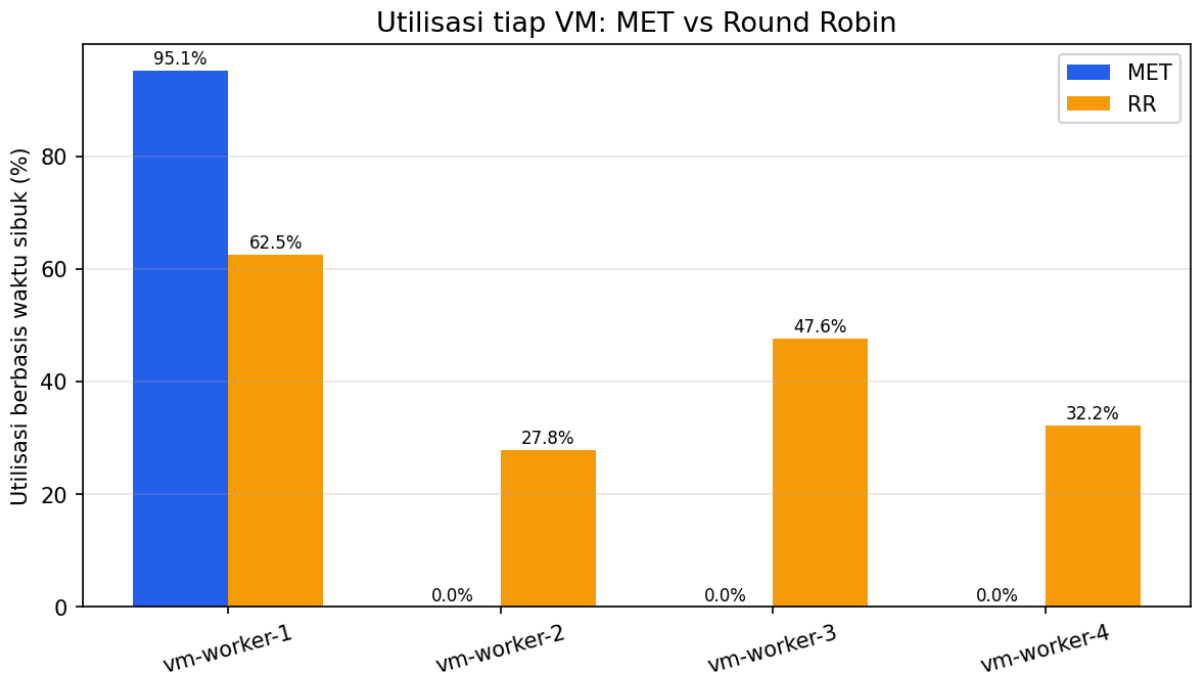
Gambar 5.2. Perbandingan throughput MET dan Round Robin.



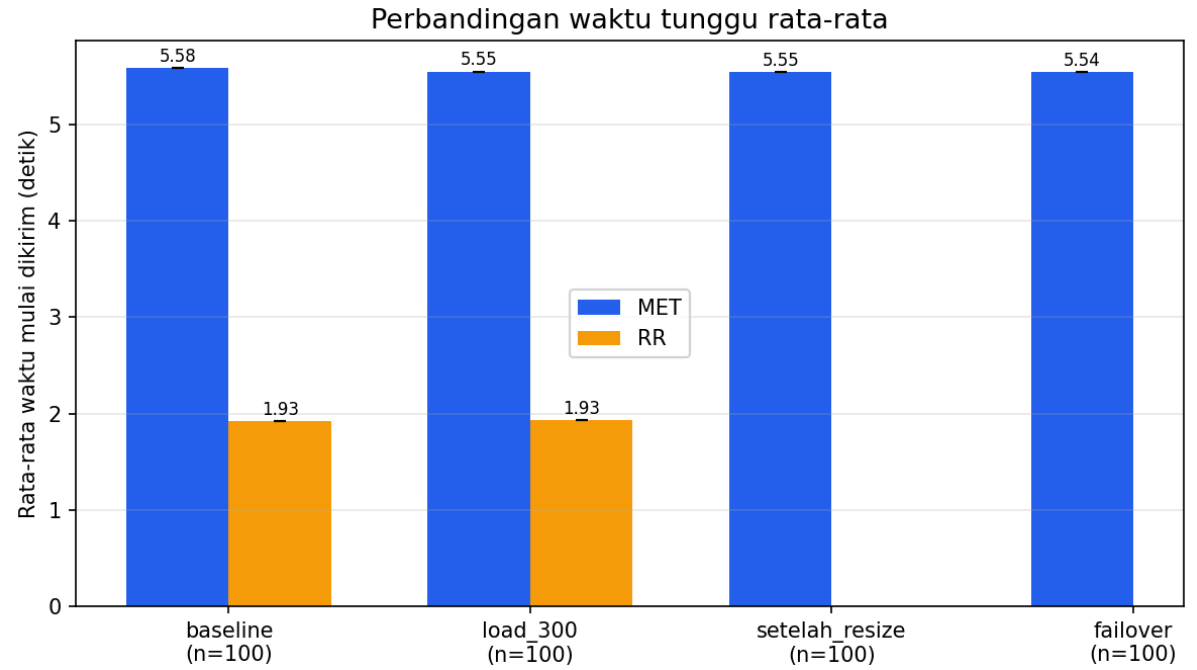
Gambar 5.3. Distribusi task pada setiap VM.



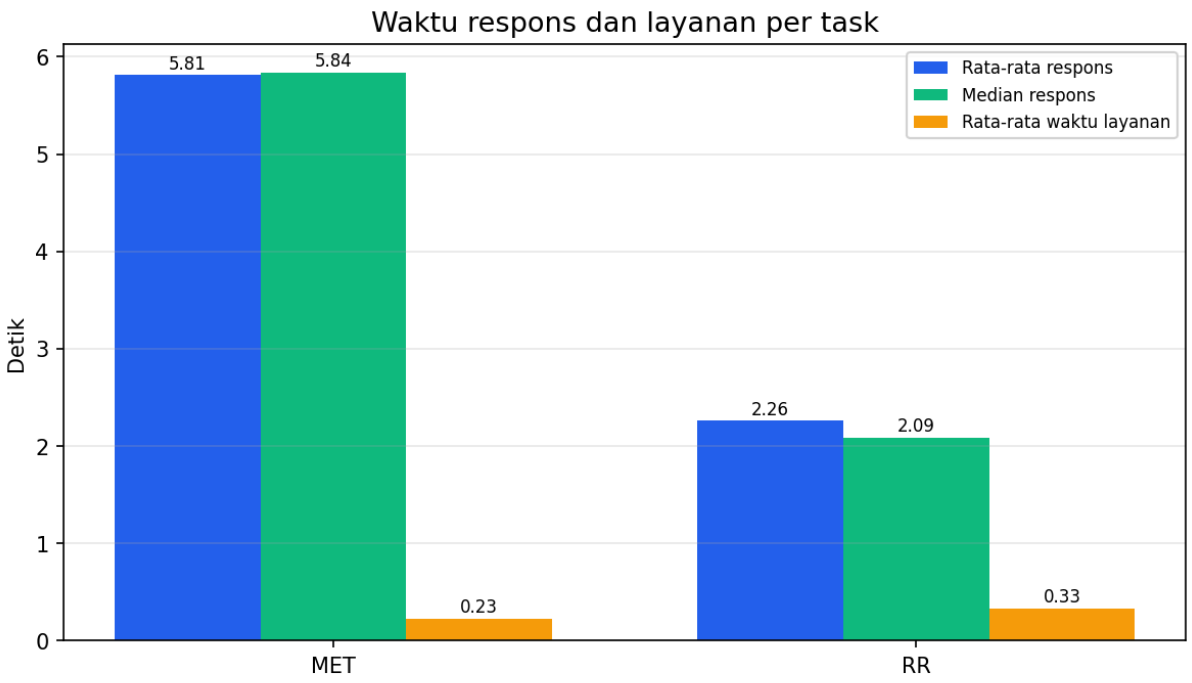
Gambar 5.4. Degree of Imbalance (DI).



Gambar 5.5. Utilisasi berbasis waktu sibuk per VM.



Gambar 5.6. Perbandingan avg_wait_s.



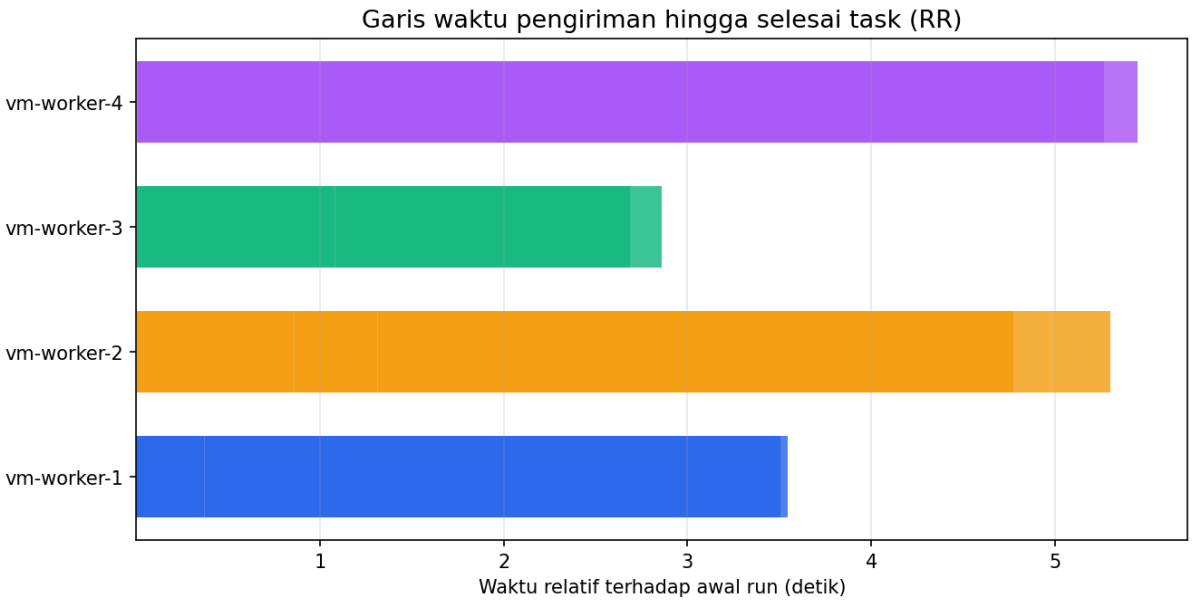
Respons = finish_s karena semua task tersedia pada t0; layanan = finish_s − send_s sesuai definisi data proyek.

Gambar 5.7. Respons dan waktu layanan per task.



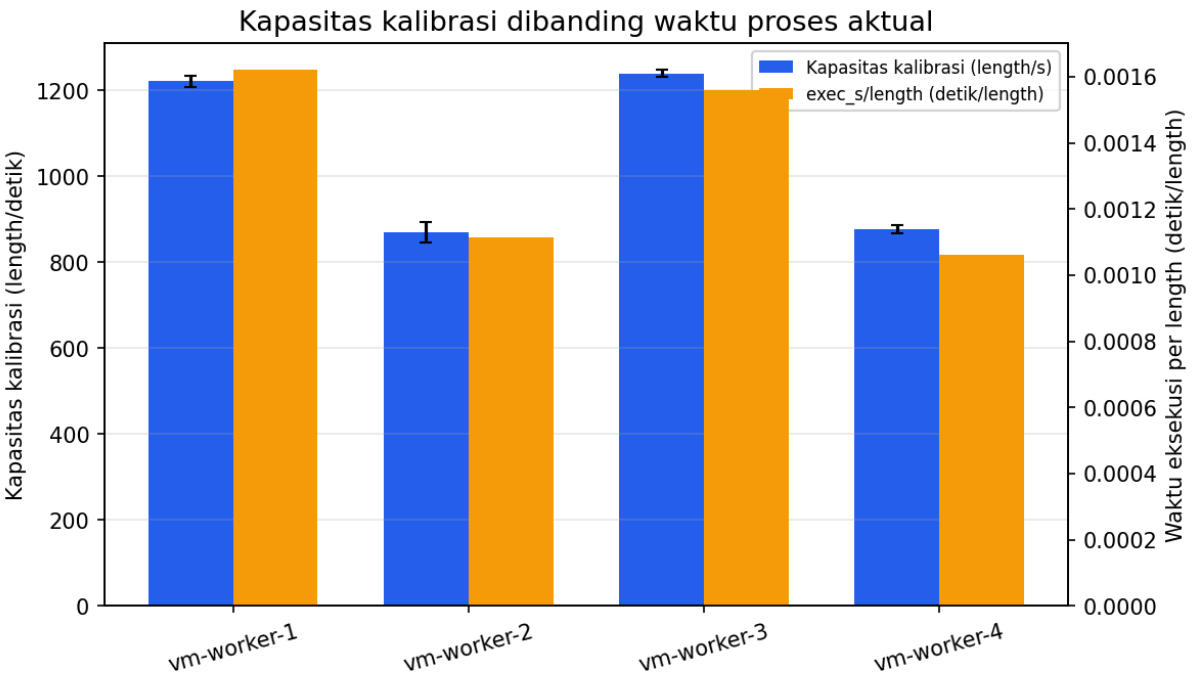
Baseline terakhir, n=100 task; batang membentang dari send_s sampai finish_s dan mencakup antrean/pengiriman.

Gambar 5.8. Garis waktu task pada run MET baseline.



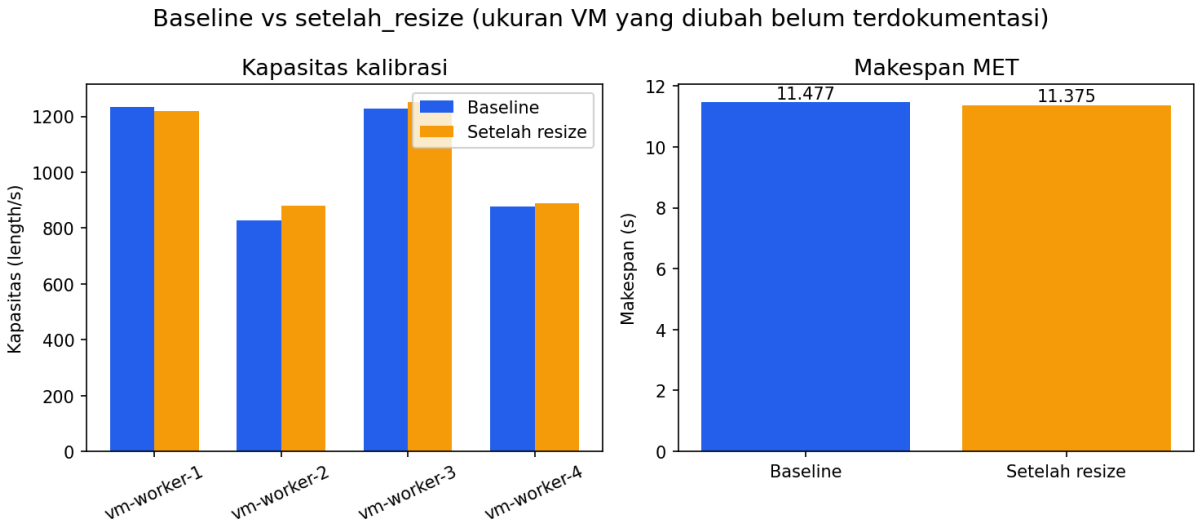
Baseline terakhir, n=100 task; batang membentang dari send_s sampai finish_s dan mencakup antrean/pengiriman.

Gambar 5.9. Garis waktu task pada run RR baseline.

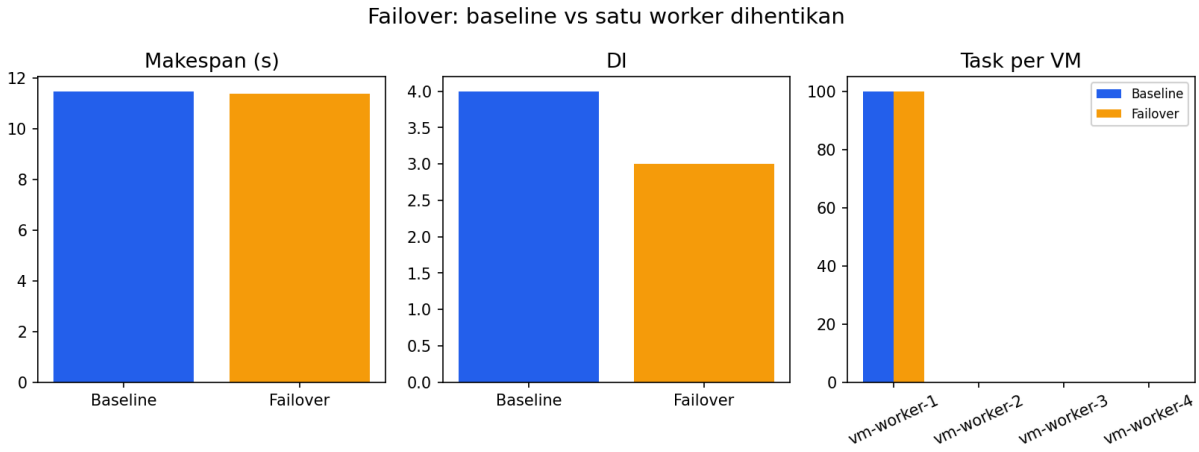


Kapasitas kalibrasi dipengaruhi waktu dinding dari master; exec_s/length dihitung dari log RR baseline terakhir yang tersedia.

Gambar 5.10. Kapasitas kalibrasi dibanding exec_s per length.



Gambar 5.11. Baseline dibanding label setelah_resize; dampak resize belum terverifikasi.



Gambar 5.12. Baseline dan failover saat worker non-target MET dihentikan.

5.2 Analisis

5.2.1 MET dibanding Round Robin

Pada baseline 100 task, RR menyelesaikan pekerjaan dalam 5,4666 detik, dibanding 11,4768 detik pada MET. Throughput RR 18,2928 task/detik, sedangkan MET 8,7132 task/detik. Log distribusi menunjukkan MET mengirim 100 task ke vm-worker-1, sedangkan RR membagi 25 task per worker. Penjelasan yang konsisten dengan kode adalah MET mengabaikan antrean dan beban yang sudah terjadwal, sementara RR menggunakan semua worker secara paralel.

5.2.2 Ketimpangan dan Utilisasi

DI MET = 4,0 sama dengan jumlah worker pada kondisi seluruh task menumpuk di satu worker. Utilisasi vm-worker-1 tercatat 95,12% dan tiga worker lainnya 0%; rata-rata empat worker menjadi 23,78%. Pada RR, DI = 0,6042 dan utilisasi rata-rata 42,51%. RR tidak mencapai utilisasi 100% karena beban length tidak identik, performa worker berbeda, dan worker yang selesai lebih awal menunggu worker lain menyelesaikan task. Transfer gambar dan waktu dinding juga memengaruhi durasi run, tetapi utilisasi proyek dihitung dari exec_s saja.

5.2.3 Kalibrasi dan Kapasitas

Kapasitas kalibrasi baseline MET tercatat sekitar 1235,49 length/detik untuk vm-worker-1 dan 1229,47 length/detik untuk vm-worker-3. Selisihnya kurang dari toleransi 5%, sehingga tie-break memilih vm-worker-1 karena urutannya lebih awal. Log RR menunjukkan total exec_s per length tidak sepenuhnya mengikuti urutan kapasitas kalibrasi. Salah satu kemungkinan adalah pengukuran kapasitas dari master ikut memasukkan transfer/latensi jaringan; ini merupakan dugaan yang perlu diuji dengan kalibrasi berbasis waktu pemrosesan worker dan pengukuran jaringan terpisah.

5.2.4 Beban 300 Task

Run berlabel load_300 mencatat n_tasks = 100, sama dengan baseline. Oleh karena itu, hasil ini tidak dapat digunakan untuk menyimpulkan throughput saat beban dinaikkan menjadi 300 task. Grafik skalabilitas sengaja tidak dibuat. Untuk pengujian valid, dataset yang berisi minimal 300 gambar perlu dibuat di folder terpisah dan run diulang dengan n_tasks yang terverifikasi pada summary.csv.

5.2.5 Resize dan Failover

Run setelah_resize mencatat makespan MET 11,3753 detik dan throughput 8,7910 task/detik, namun ukuran VM sebelum/sesudah tidak tercatat dan kapasitas hasil kalibrasi hampir tidak berubah. Dampak resize belum dapat disimpulkan. Pada run failover, vm-worker-4 dihentikan; MET tetap menaruh seluruh task di vm-worker-1, makespan 11,3772 detik dan DI 3,0 karena hanya tiga worker sehat. Ini menguji hilangnya worker non-target, bukan skenario paling kritis saat vm-worker-1 sebagai target MET dihentikan. Uji failover lokal terhadap target MET memiliki bukti di repo, tetapi tidak menggantikan uji target di Azure.

5.3 Keterbatasan

- Baseline yang dibahas hanya satu run per algoritma; tidak cukup untuk menyatakan hasil signifikan secara statistik.
- Run berlabel load_300 masih berisi 100 task; belum ada hasil valid untuk 300 atau 500 task.
- Ukuran SKU VM dan dokumentasi resize belum tersedia dalam artefak yang diberikan.
- Failover Azure belum menghentikan VM target MET; uji Azure hanya menghentikan vm-worker-4.
- Utilisasi yang dilaporkan adalah utilisasi berbasis waktu sibuk, bukan CPU% dari Azure Monitor.
- Worker diakses melalui IP publik pada port 8000; ini berbeda dari rencana IP privat dan memerlukan verifikasi aturan keamanan.
- Run awal dengan tiga worker dikecualikan dari baseline empat worker; hasilnya tidak boleh digabung tanpa penjelasan.
- Peringatan master menyebut estimasi makespan berada di luar rentang 30–180 detik; workload saat ini hanya berlangsung sekitar 5–12 detik.

5.4 Kesimpulan dan Saran

MET berhasil dijalankan pada Azure dan menunjukkan pola yang sama seperti simulasi sebelumnya: seluruh task cenderung diarahkan ke VM yang dianggap memiliki kapasitas terbaik, sehingga DI mencapai jumlah worker. Namun, pada baseline yang tersedia, Round Robin menghasilkan makespan lebih rendah dan throughput lebih tinggi karena

memanfaatkan empat worker. Hasil ini tidak membuktikan RR selalu lebih baik pada semua workload; kesimpulan dibatasi pada konfigurasi, dataset, dan implementasi yang diuji.

Lampiran A. Cara Membuat Grafik

Dari direktori repo, jalankan:

python plot_results.py --results results

Dependensi: pandas, matplotlib, numpy. Skrip menghasilkan results/figures/, results/hasil_ringkasan.md, results/hasil_agregat.csv, dan results/catatan_grafik.md. Untuk memasukkan run non-failover dengan kurang dari empat worker, gunakan --include-partial. Jangan menafsirkan label sebagai jumlah task; selalu gunakan kolom n_tasks.